
Indian Institute of Technology, Patna
Department of Computer Science and Engineering
CS3101: Operating System
Lab Assignment 5: Process Chains, Multi-Threading, and
Inter-Process Communication

Question 1: Bidirectional Inter-Process Communication Using Ordinary Pipes

Problem Statement

Design and implement a program in C that demonstrates bidirectional inter-process communication between a parent process and a child process using ordinary unnamed pipes.

The program must create exactly two ordinary pipes:

- A `Parent-to-Child` pipe for sending requests.
- A `Child-to-Parent` pipe for returning processed results.

The parent process must repeatedly accept a line of text from the user and send it to the child process. The child must process each received line and return the processed result to the parent process.

Processing Performed by the Child

For every message received from the parent, the child process must perform the following operations:

1. Reverse the case of every alphabetic character:
 - Convert every uppercase letter to lowercase.
 - Convert every lowercase letter to uppercase.
2. Reverse the order of the words without reversing the characters within an individual word.
3. Calculate the following statistics from the original message:
 - Number of uppercase letters.

- Number of lowercase letters.
- Number of digits.
- Number of words.

The child process must send both the transformed string and the calculated statistics back to the parent process.

Example

If the parent process sends the following message:

```
Hi There 2026
```

the child process must return the following transformed message:

```
2026 tHERE hI
```

The statistics calculated from the original message must be:

```
Uppercase letters : 2
Lowercase letters : 5
Digits            : 4
Words             : 3
```

Repeated Communication and Termination

The parent process must continue accepting and sending messages until the user enters the following termination command:

```
exit
```

When the user enters `exit`, the following operations must be performed:

1. The parent sends the termination command to the child.
2. The child recognizes the command and terminates gracefully without processing it as an ordinary message.
3. The child closes all its remaining pipe descriptors before terminating.
4. The parent closes all its remaining pipe descriptors.
5. The parent waits for the child using `wait()` or `waitpid()`.
6. The parent prints the termination status of the child before terminating.

Mandatory Requirements

The implementation must satisfy all the following requirements:

- Use the `pipe()`, `fork()`, `read()`, and `write()` system calls.
- Use exactly two ordinary unnamed pipes.
- Do not use named pipes, sockets, shared memory, temporary files, `popen()`, or `system()`.
- Do not create any additional child processes.
- The parent and child must close every unused pipe endpoint immediately after `fork()`.
- The program must correctly process spaces, digits, punctuation marks, and mixed-case alphabetic characters.
- Each input line may contain a maximum of 256 characters, including spaces.
- Empty input lines must be handled without terminating the program.
- The return values of all relevant system calls must be checked. Appropriate error messages must be displayed when a system call fails.
- The parent process must not perform the required string transformation or calculate the statistics.
- The child process must not read input directly from the terminal.
- The program must not assume that a single invocation of `read()` or `write()` transfers the complete message.

Message-Framing Requirement

Ordinary pipes provide a byte-stream communication mechanism and do not preserve message boundaries. Therefore, every request and response must contain:

1. The length of the data.
2. The actual data.

Students must implement helper functions equivalent to the following:

```
1 ssize_t read_full(int fd, void *buffer, size_t count);  
2  
3 ssize_t write_full(int fd, const void *buffer,  
4                   size_t count);
```

The `read_full()` function must continue reading until:

- the requested number of bytes has been received,
- an end-of-file condition is encountered, or
- an error occurs.

Similarly, the `write_full()` function must continue writing until the complete data has been transferred or an error occurs.

Transfer of Statistics

The child must store the calculated statistics in a structure similar to the following:

```
1 typedef struct {  
2     int uppercase_count;  
3     int lowercase_count;  
4     int digit_count;  
5     int word_count;  
6 } Statistics;
```

The structure must be populated by the child process and transferred to the parent process through the Child-to-Parent pipe.

The statistics must always be calculated from the original message received from the parent, not from the transformed message.

Sample Interaction

Parent PID: 4120

Child PID : 4121

Enter a message: Hi There 2026

Result received from child:

Transformed message : 2026 tHERE hI

Uppercase letters : 2

Lowercase letters : 5

Digits : 4

Words : 3

Enter a message: Operating Systems LAB

Result received from child:

Transformed message : lab SYSTEMS oPERATING

Uppercase letters : 5

Lowercase letters : 14

Digits : 0

Words : 3

Enter a message: exit

Child terminated normally with exit status 0.

Parent terminating.

Compilation Instructions

The program must compile without warnings using the following command:

```
gcc -Wall -Wextra -pedantic <RollNo>_q1.c -o q1
```

Submission Instructions

Submit the following items:

1. The C source file named:

`<RollNo>_q1.c`

2. A screenshot showing successful compilation without warnings using the command specified above.
3. Sample input/output screenshots demonstrating:
 - An input containing multiple words and spaces.
 - An input containing letters, digits, and punctuation marks.
 - Multiple request–response interactions between the parent and child.
 - Graceful termination using the `exit` command.

Question 2A: Fibonacci Sequence and Process Status Using `fork()`

Problem Statement

Write a C program that uses the `fork()` system call to generate and analyse a Fibonacci sequence.

The number of terms, n , must be supplied as a command-line argument.

Child-Process Responsibilities

The child process must perform the following operations:

1. Generate the first n terms of the Fibonacci sequence.
2. Display its process ID and the process ID of its parent using `getpid()` and `getppid()`.
3. Print the generated Fibonacci sequence.
4. Count the number of even-valued terms in the generated sequence.
5. Terminate using `exit()` or `_exit()`, returning the even-term count as its exit status.

Parent-Process Responsibilities

The parent process must perform the following operations:

1. Validate the command-line argument before calling `fork()`.
2. Display its process ID and the process ID of the newly created child.
3. Wait for the child process using `waitpid()`.
4. Determine whether the child:
 - terminated normally, or
 - was terminated by a signal.
5. If the child terminated normally, obtain and display the even-term count using:
 - `WIFEXITED()`
 - `WEXITSTATUS()`
6. Terminate only after reporting the termination status of the child.

Input Constraints

The program must satisfy the following input-validation requirements:

- Exactly one command-line argument must be supplied.
- The argument must represent a valid integer.
- The value of n must satisfy:

$$1 \leq n \leq 40$$

- Invalid inputs such as 5abc, -3, 0, or an empty argument must be rejected.
- The `atoi()` function must not be used for input validation. Use `strtol()` or an equivalent safe method.

Example Execution

```
$ ./q2a 8
```

```
Parent process:
```

```
Parent PID : 4200
```

```
Child PID  : 4201
```

```
Child process:
```

```
Child PID  : 4201
```

```
Parent PID : 4200
```

```
Fibonacci sequence: 0 1 1 2 3 5 8 13
```

```
Even-valued terms : 3
```

```
Parent process:
```

```
Child 4201 terminated normally.
```

```
Even-term count returned by child: 3
```

```
Parent terminating.
```

Mandatory Requirements

- Use `fork()`, `getpid()`, `getppid()`, and `waitpid()`.
- The Fibonacci sequence must be generated only by the child process.

- The parent process must not regenerate the Fibonacci sequence.
- The parent must inspect the child's termination status using the appropriate status macros.
- All relevant system and library calls must be checked for errors.
- Do not use pipes, shared memory, files, threads, sockets, or global variables for communication.
- Ensure that buffered output is not unintentionally duplicated after calling `fork()`.

Submission Instructions

Submit the following items:

1. The C source file named:

`<RollNo>_q2a.c`

2. A screenshot showing successful compilation without warnings using:

```
gcc -Wall -Wextra -pedantic <RollNo>_q2a.c -o q2a
```

3. A sample input/output screenshot demonstrating:

- successful execution for a valid value of n ,
- the PID information of both processes,
- the Fibonacci sequence generated by the child,
- the even-term count received through the exit status, and
- rejection of at least one invalid command-line argument.

Question 2B: Coordinated Sequence Processing Using POSIX Shared Memory

Problem Statement

Extend Question 2A to implement coordinated processing of a Fibonacci sequence using POSIX shared memory and two child processes.

The program must create the following processes:

1. A **generator child** that generates the Fibonacci sequence.
2. An **analyser child** that analyses the sequence produced by the generator.
3. The original **parent process** that displays the final result and releases all shared resources.

The required execution order is:

Generator Child $\longrightarrow$ Analyser Child $\longrightarrow$ Parent

The parent process must enforce this execution order using `waitpid()`.

Shared-Memory Structure

Use the following definitions for the shared-memory segment:

```
1 #define MAX_SEQUENCE 40
2
3 typedef struct {
4     long fib_sequence[MAX_SEQUENCE];
5     int sequence_size;
6
7     long sequence_sum;
8     int even_count;
9     int odd_count;
10
11     int generator_completed;
12     int analyser_completed;
13 } SharedData;
```

Parent-Process Responsibilities

The parent process must perform the following operations:

1. Accept n , the required number of Fibonacci terms, as a command-line argument.
2. Validate that n is an integer satisfying:

$$1 \leq n \leq \text{MAX_SEQUENCE}$$

3. Create a uniquely named POSIX shared-memory object using `shm_open()`.
4. Set the size of the shared-memory object using `ftruncate()`.
5. Map the shared-memory object into its address space using `mmap()`.
6. Initialize the complete shared-memory structure before creating either child process.
7. Fork the generator child.
8. Wait for the generator child using `waitpid()`.
9. Verify that the generator child terminated successfully.
10. Fork the analyser child only after the generator has completed successfully.
11. Wait for the analyser child using `waitpid()`.
12. Verify that both completion flags have been set correctly.
13. Display:
 - the complete Fibonacci sequence,
 - the sum of all sequence terms,
 - the number of even-valued terms, and
 - the number of odd-valued terms.
14. Unmap the shared-memory region using `munmap()`.
15. Close the shared-memory descriptor using `close()`.
16. Remove the shared-memory object using `shm_unlink()`.

Generator-Child Responsibilities

The generator child must perform the following operations:

1. Use the shared-memory mapping inherited from the parent process.
2. Generate the first n terms of the Fibonacci sequence.
3. Store the generated terms in `fib_sequence[]`.

4. Store n in `sequence_size`.
5. Set `generator_completed` to 1 only after the complete sequence has been generated.
6. Display its PID and indicate that sequence generation has been completed.
7. Unmap the shared-memory region using `munmap()`.
8. Close the inherited shared-memory descriptor.
9. Terminate successfully.

The generator child must not calculate the sequence sum or the even and odd term counts.

Analyser-Child Responsibilities

The analyser child must perform the following operations:

1. Use the shared-memory mapping inherited from the parent process.
2. Verify that `generator_completed` is equal to 1.
3. Read the generated sequence from `fib_sequence[]`.
4. Calculate:
 - the sum of all terms,
 - the number of even-valued terms, and
 - the number of odd-valued terms.
5. Store the calculated results in their corresponding shared-memory fields.
6. Set `analyser_completed` to 1 only after completing the analysis.
7. Display its PID and indicate that sequence analysis has been completed.
8. Unmap the shared-memory region using `munmap()`.
9. Close the inherited shared-memory descriptor.
10. Terminate successfully.

The analyser child must not regenerate or modify the Fibonacci sequence.

Synchronization Requirement

The parent must enforce the following sequence of operations:

1. Create the generator child.
2. Wait for the generator child to terminate.
3. Verify the generator's termination status.
4. Create the analyser child.
5. Wait for the analyser child to terminate.
6. Verify the analyser's termination status.
7. Print the final sequence and calculated statistics.

The parent must not use `sleep()`, `usleep()`, or busy waiting for synchronization.

Shared-Memory Naming Requirement

The parent must create a unique shared-memory name based on its process ID to avoid collisions between simultaneous executions.

The shared-memory name should have a form similar to:

```
/fibonacci_<parent-pid>
```

For example, if the parent PID is 5100, the shared-memory name may be:

```
/fibonacci_5100
```

Example Execution

```
$ ./q2b 8
```

```
Parent PID: 5100
```

```
Shared-memory object created.
```

```
Generator child PID: 5101
```

```
Generator completed successfully.
```

```
Analyser child PID: 5102
```

```
Analyser completed successfully.
```

```
Final result printed by parent:
```

```
Fibonacci sequence : 0 1 1 2 3 5 8 13
Sequence sum       : 33
Even-valued terms  : 3
Odd-valued terms   : 5
```

```
Shared-memory object removed.
Parent terminating.
```

Error Handling and Cleanup Requirements

The program must check and correctly handle failures of:

- `strtol()`
- `shm_open()`
- `ftruncate()`
- `mmap()`
- `fork()`
- `waitpid()`
- `munmap()`
- `close()`
- `shm_unlink()`

If an error occurs after the shared-memory object has been created, the parent must still perform all applicable cleanup operations before terminating.

If the generator child fails, the parent must not create the analyser child.

Mandatory Requirements

- Use POSIX shared memory, not System V shared memory.
- Use `shm_open()`, `ftruncate()`, `mmap()`, `munmap()`, and `shm_unlink()`.
- Use exactly two child processes.
- Do not use pipes, regular files, sockets, message queues, global arrays, or threads.
- Do not use `sleep()` or busy waiting for synchronization.
- The parent must be the only process that prints the final sequence and statistics.

- The generator child must only generate the sequence.
- The analyser child must only calculate the statistics.
- All processes must close and unmap the resources that they no longer require.
- The parent must remove the POSIX shared-memory object before terminating.

Submission Instructions

Submit the following items:

1. The C source file named:

`<RollNo>_q2b.c`

2. A screenshot showing successful compilation without warnings using:

```
gcc -Wall -Wextra -pedantic <RollNo>_q2b.c -o q2b
```

On Linux systems that require the real-time library, compile using:

```
gcc -Wall -Wextra -pedantic <RollNo>_q2b.c -o q2b -lrt
```

3. Sample input/output screenshots demonstrating:

- successful sequence generation,
- execution of both child processes,
- correct process ordering,
- the final sequence and statistics printed by the parent,
- successful shared-memory cleanup, and
- rejection of at least one invalid command-line argument.

Question 3: Multi-Process, Multi-Threaded Newsroom

High-Level Goal

Design and implement a C program that simulates a newsroom consisting of one Editor process and multiple Journalist processes.

The Editor coordinates the publication of a fixed number of articles. Each Journalist operates as an independent process and internally uses multiple threads to research, write, and submit articles through a synchronized pipeline.

This question combines the following operating-system concepts:

- Process creation
- Program replacement
- Ordinary pipes
- I/O multiplexing
- POSIX threads
- Mutexes
- Condition variables
- Bounded buffers
- Graceful process and thread termination

Program Files

The solution must consist of the following files:

```
<RollNo>_editor.c  
<RollNo>_journalist.c  
Makefile
```

The two C programs must be compiled into separate executable files:

```
editor  
journalist
```

Command-Line Interface

The newsroom must be started using:

```
./editor <number-of-journalists> <publication-target>
```

For example:

```
./editor 3 10
```

In this example:

- 3 is the number of Journalist processes.
- 10 is the total number of articles the Editor must publish.

The program must enforce:

$$1 \leq N \leq 10$$

and

$$1 \leq M \leq 100,$$

where:

- N is the number of Journalists.
- M is the publication target.

The arguments must be validated using `strtol()`. The use of `atoi()` for input validation is not permitted.

Process Architecture

The simulation must contain one Editor process and N Journalist processes.

Editor Process

The Editor is the original parent process. It must:

1. Validate the command-line arguments.

2. Create communication pipes for every Journalist.
3. Create N Journalist child processes using `fork()`.
4. Replace each child process with the `journalist` executable using `execvp()`.
5. Monitor all article pipes using `select()`.
6. Publish exactly M articles.
7. Send a termination command to every Journalist.
8. Collect and display final statistics.
9. Wait for every Journalist using `waitpid()`.
10. Close all pipe descriptors before terminating.

Journalist Processes

Each Journalist must be created using the following sequence:

$$\text{fork}() \longrightarrow \text{execvp}()$$

Every Journalist must execute the same `journalist` program but must receive a unique numeric ID.

For example:

```
Journalist 1
Journalist 2
Journalist 3
```

Each Journalist must receive the following through its command-line arguments:

- Its Journalist ID
- Its article-pipe write descriptor
- Its control-pipe read descriptor

Pipe Architecture

The Editor must create exactly two ordinary pipes for each Journalist.

For Journalist i , the communication directions are:

Article pipe: `Journalist i → Editor`

Control pipe: `Editor → Journalist i`

Therefore, for N Journalists, the Editor must create:

$$2N$$

ordinary pipes.

Article Pipe

The Submitter thread of a Journalist must use its article pipe to send completed articles to the Editor.

Control Pipe

The Editor must use a Journalist's control pipe to send a shutdown command after the publication target has been reached.

All processes must close unused pipe endpoints immediately after `fork()`.

Before calling `execvp()`, every Journalist child must also close all pipe descriptors belonging to other Journalists.

Thread Architecture Within Each Journalist

Each Journalist process must create exactly three worker threads:

1. Researcher thread
2. Writer thread
3. Submitter thread

The threads must form the following concurrent producer–consumer pipeline:

`Researcher → Topic Queue → Writer → Article Queue → Submitter`

Researcher Thread

The Researcher thread must:

1. Generate story topics.
2. Assign a sequence number to every topic.
3. Insert generated topics into a bounded topic queue.
4. Wait when the topic queue is full.
5. Terminate when the shared shutdown flag is set.

A generated topic may have the following form:

```
Story 4 from Journalist 2
```

Writer Thread

The Writer thread must:

1. Remove topics from the topic queue.
2. Wait when the topic queue is empty.
3. Convert each topic into a completed article.
4. Insert the completed article into a bounded article queue.
5. Wait when the article queue is full.
6. Terminate gracefully after shutdown.

A completed article may have the following form:

```
Article on Story 4 from Journalist 2
```

Submitter Thread

The Submitter thread must:

1. Remove completed articles from the article queue.
2. Wait when the article queue is empty.
3. Send completed articles to the Editor through the Journalist's article pipe.
4. Check the control pipe for a shutdown command.
5. Set the Journalist's shared shutdown flag after receiving the command.

6. Wake any threads waiting on condition variables.
7. Terminate after completing the required cleanup.

Bounded-Buffer Requirement

Each Journalist must maintain two bounded circular queues:

- A topic queue
- An article queue

Use the following queue capacity:

```
1 #define QUEUE_CAPACITY 3
```

The queues must not be implemented as unbounded linked lists.

Each queue must maintain appropriate state, including:

```
1 int front;  
2 int rear;  
3 int count;
```

The following blocking requirements apply:

- The Researcher must block when the topic queue is full.
- The Writer must block when the topic queue is empty.
- The Writer must block when the article queue is full.
- The Submitter must block when the article queue is empty.

Busy waiting is not permitted.

Thread-Synchronization Requirements

Each Journalist must use POSIX threads.

The following functions are mandatory:

- `pthread_create()`
- `pthread_join()`
- `pthread_mutex_lock()`

- `pthread_mutex_unlock()`
- `pthread_cond_wait()`
- `pthread_cond_signal()` or `pthread_cond_broadcast()`

Shared queue state and shutdown state must be protected using mutexes.

Condition variables must represent conditions such as:

- Topic queue is not empty
- Topic queue is not full
- Article queue is not empty
- Article queue is not full

The use of `sleep()` or `usleep()` as a replacement for proper synchronization is not permitted.

Every condition-variable wait must be enclosed in a `while` loop, rather than an `if` statement, to handle spurious wakeups correctly.

Article Message Format

Journalists must not send unstructured or variable-length text directly through the article pipes.

Each completed article must be transferred using a structure similar to:

```
1 #define TOPIC_LENGTH    100
2 #define ARTICLE_LENGTH 200
3
4 typedef struct {
5     int  journalist_id;
6     int  article_number;
7     char topic[TOPIC_LENGTH];
8     char article_text[ARTICLE_LENGTH];
9 } ArticleMessage;
```

Each article message must contain:

- Journalist ID
- Journalist-local article number
- Topic

- Completed article text

The size of `ArticleMessage` must not exceed `PIPE_BUF`.

Each structure must be sent using a complete-write helper function. The implementation must not assume that one invocation of `write()` always transfers the complete structure.

Control Message Format

The Editor must send a structured shutdown command through each control pipe.

Use a structure similar to:

```
1 typedef enum {  
2     CONTROL_STOP = 1  
3 } ControlType;  
4  
5 typedef struct {  
6     ControlType type;  
7 } ControlMessage;
```

When the Submitter receives `CONTROL_STOP`, it must:

1. Lock the appropriate mutex.
2. Set the shared shutdown flag.
3. Broadcast all relevant condition variables.
4. Unlock the mutex.
5. Stop submitting new articles.
6. Allow all Journalist threads to terminate gracefully.

The use of `SIGKILL` for ordinary termination is prohibited.

Detailed Editor Responsibilities

Initialization

The Editor must:

1. Validate N and M .
2. Dynamically allocate the required pipe arrays.

3. Create two pipes for every Journalist.
4. Create all Journalist processes using `fork()`.
5. Use `execvp()` to execute the Journalist program.
6. Store the PID of every Journalist.
7. Close every unused pipe endpoint.

Monitoring Article Pipes

The Editor must use `select()` to monitor all Journalist article pipes simultaneously.

The Editor must not:

- Read the article pipes sequentially in a fixed order.
- Use one Editor thread per Journalist.
- Use busy waiting.
- Use `sleep()` to periodically check the pipes.

Before every call to `select()`, the Editor must rebuild the descriptor set because `select()` modifies it.

Publishing Articles

Whenever an article becomes available, the Editor must:

1. Read one complete `ArticleMessage`.
2. Validate the Journalist ID.
3. Assign a global publication number.
4. Print the published article.
5. Increment the corresponding Journalist's publication count.

The Editor must publish exactly M articles.

Any additional article received after the publication target has been reached must not be counted as published.

Newsroom Shutdown

After publishing M articles, the Editor must:

1. Send `CONTROL_STOP` to every Journalist.
2. Close every control-pipe write endpoint.
3. Continue closing or draining article pipes as necessary so that no Journalist remains blocked indefinitely.
4. Wait for all Journalist processes using `waitpid()`.
5. Report whether every Journalist terminated normally.
6. Print the final newsroom summary.
7. Release all dynamically allocated resources.
8. Terminate only after all child processes have been collected.

Detailed Journalist Responsibilities

Each Journalist process must:

1. Validate its command-line arguments.
2. Create and initialize both bounded queues.
3. Initialize all mutexes and condition variables.
4. Create exactly three threads.
5. Join all three threads using `pthread_join()`.
6. Close its article and control pipe descriptors.
7. Destroy all mutexes and condition variables.
8. Release all dynamically allocated memory.
9. Print a termination message.
10. Exit with `EXIT_SUCCESS` only if all operations completed successfully.

If any thread cannot be created, the Journalist must:

- Set the shutdown flag.
- Wake any already-created threads.

- Join all successfully created threads.
- Clean up all initialized resources.
- Exit with `EXIT_FAILURE`.

Required Final Statistics

After all Journalist processes have terminated, the Editor must print:

1. Total number of articles published.
2. Number of articles published from each Journalist.
3. Journalist with the highest number of published articles.
4. Whether every Journalist terminated normally.

If multiple Journalists have the same highest publication count, all such Journalist IDs must be displayed.

An example summary is:

Newsroom summary:

Total articles published : 10

Journalist 1 : 3

Journalist 2 : 4

Journalist 3 : 3

Highest contribution : Journalist 2

All journalists exited normally.

Error-Handling Requirements

The programs must check failures of all relevant functions, including:

- `strtol()`
- `malloc()` or `calloc()`
- `pipe()`
- `fork()`
- `execvp()`
- `select()`

- `read()`
- `write()`
- `close()`
- `waitpid()`
- `pthread_create()`
- `pthread_join()`
- `pthread_mutex_init()`
- `pthread_mutex_destroy()`
- `pthread_cond_init()`
- `pthread_cond_destroy()`

Interrupted system calls must be handled appropriately.

A failure in one child process must not cause the Editor to wait indefinitely.

Prohibited Techniques

The following techniques are not permitted:

- Named pipes
- Shared memory
- Message queues
- Sockets
- Temporary files
- Global files for communication
- Busy waiting
- Synchronization using `sleep()`
- Creating one Editor thread per Journalist
- Using `system()` to execute the Journalist program
- Using `SIGKILL` for normal termination

Sample Execution

```
$ ./editor 3 10
```

```
Editor: Newsroom opened.
```

```
Editor PID: 6100
```

```
Editor: Waiting for 10 articles from 3 journalists.
```

```
Journalist 1, Researcher:
```

```
Generated "Story 1 from Journalist 1"
```

```
Journalist 2, Researcher:
```

```
Generated "Story 1 from Journalist 2"
```

```
Journalist 1, Writer:
```

```
Completed "Article on Story 1 from Journalist 1"
```

```
Journalist 3, Researcher:
```

```
Generated "Story 1 from Journalist 3"
```

```
Journalist 1, Submitter:
```

```
Submitted article 1.
```

```
Editor:
```

```
Published [1/10] from Journalist 1:
```

```
"Article on Story 1 from Journalist 1"
```

```
Journalist 2, Writer:
```

```
Completed "Article on Story 1 from Journalist 2"
```

```
Journalist 2, Submitter:
```

```
Submitted article 1.
```

```
Editor:
```

```
Published [2/10] from Journalist 2:
```

```
"Article on Story 1 from Journalist 2"
```

```
...
```

Editor:

Publication target reached. Sending shutdown commands.

Journalist 2:

Shutdown command received.

Journalist 1:

Shutdown command received.

Journalist 3:

Shutdown command received.

Journalist 2:

All worker threads terminated. Exiting.

Journalist 1:

All worker threads terminated. Exiting.

Journalist 3:

All worker threads terminated. Exiting.

Editor:

Journalist 1 terminated normally.

Journalist 2 terminated normally.

Journalist 3 terminated normally.

Newsroom summary:

Total articles published : 10

Journalist 1 : 3

Journalist 2 : 4

Journalist 3 : 3

Highest contribution : Journalist 2

All journalists exited normally.

Editor: Newsroom closed.

Because the program executes concurrently, the exact order of Journalist messages may differ between executions.

Makefile Requirements

Students must provide a `Makefile` that supports the following commands:

```
make
make clean
```

Running `make` must generate:

```
editor
journalist
```

Both programs must be compiled using:

```
-Wall -Wextra -pedantic
```

The `Journalist` program must additionally be linked using:

```
-pthread
```

A suitable `Makefile` structure is:

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -pedantic
3
4 all: editor journalist
5
6 editor: <RollNo>_editor.c
7     $(CC) $(CFLAGS) <RollNo>_editor.c -o editor
8
9 journalist: <RollNo>_journalist.c
10    $(CC) $(CFLAGS) <RollNo>_journalist.c \
11        -o journalist -pthread
12
13 clean:
14    rm -f editor journalist
```

The command lines inside `Makefile` recipes must begin with a tab character, not spaces.